МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №1

Специальность системы управления информацией

Выполнил

К. В. Пух

студент группы АС-68/24

Проверил

А. А. Крощенко,

доцент Кафедры ИИТ,

«__» _____2026 г.

Брест 2026

Цель: Получить практические навыки работы с данными с использованием библиотек Pandas для манипуляции и Matplotlib для визуализации.

Научиться выполнять основные шаги предварительной обработки данных, такие как очистка, нормализация и работа с различными типами признаков.

Общее задание:

1. Загрузить предложенный набор данных (по вариантам) в DataFrame библиотеки Pandas.

   Код программы:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("auto-mpg.csv", na_values=["?"])
print(df.head())
```

2. Провести исследовательский анализ: изучить типы данных, количество пропусков, основные статистические показатели (среднее, медиана, стандартное отклонение).

   Код программы:

```
df.info()
print(df.isnull().sum())
print(df.describe())
print(df.median(numeric_only=True))
print(df.std(numeric_only=True))
```

3. Обработать пропущенные значения (например, заполнить средним значением или удалить строки/столбцы).

   Код программы:

```
df["horsepower"] = pd.to_numeric(df["horsepower"],
errors="coerce")
mean_hp = df["horsepower"].mean()
df["horsepower"] = df["horsepower"].fillna(mean_hp)
print("Пропусков после обработки:",
df["horsepower"].isnull().sum())
```

4. Преобразовать категориальные признаки в числовые с помощью метода One-Hot Encoding. – в задании по вариантам

```
origin_encoded = pd.get_dummies(df["origin"], prefix="origin",
dtype=int)
df = pd.concat([df, origin_encoded], axis=1)
    print(origin_encoded.head())
```

5. Выполнить нормализацию или стандартизацию числовых признаков. – в задании по вариантам

```
numeric_cols = ["mpg", "cylinders", "displacement", "horsepower",
"weight", "acceleration", "age"]
normalized = df[numeric_cols].copy()
for col in numeric_cols:
      min_val = normalized[col].min()
      max_val = normalized[col].max()
      normalized[col] = (normalized[col] - min_val) / (max_val -
min_val)
print(normalized.head())
```

6. Построить несколько графиков для визуализации данных (гистограммы, диаграммы рассеяния) и сделать выводы о зависимостях между признаками. Код программы:

```
# Диаграмма рассеяния mpg от weight
plt.scatter(df["weight"], df["mpg"], alpha=0.6)
plt.savefig("01_weight_mpg.png")

# Столбчатая диаграмма по числу цилиндров
counts = df["cylinders"].value_counts().sort_index()
plt.bar(counts.index.astype(str), counts.values)
plt.savefig("02_cylinders.png")
```

Задания по варианту №7 (Auto MPG)

1. Загрузка данных и вывод информации
   Код программы:

```
df = pd.read_csv("auto-mpg.csv", na_values=["?"])
print(df.head())
df.info()
print(df.isnull().sum())
```

2. Преобразование horsepower в числовой формат и заполнение пропусков

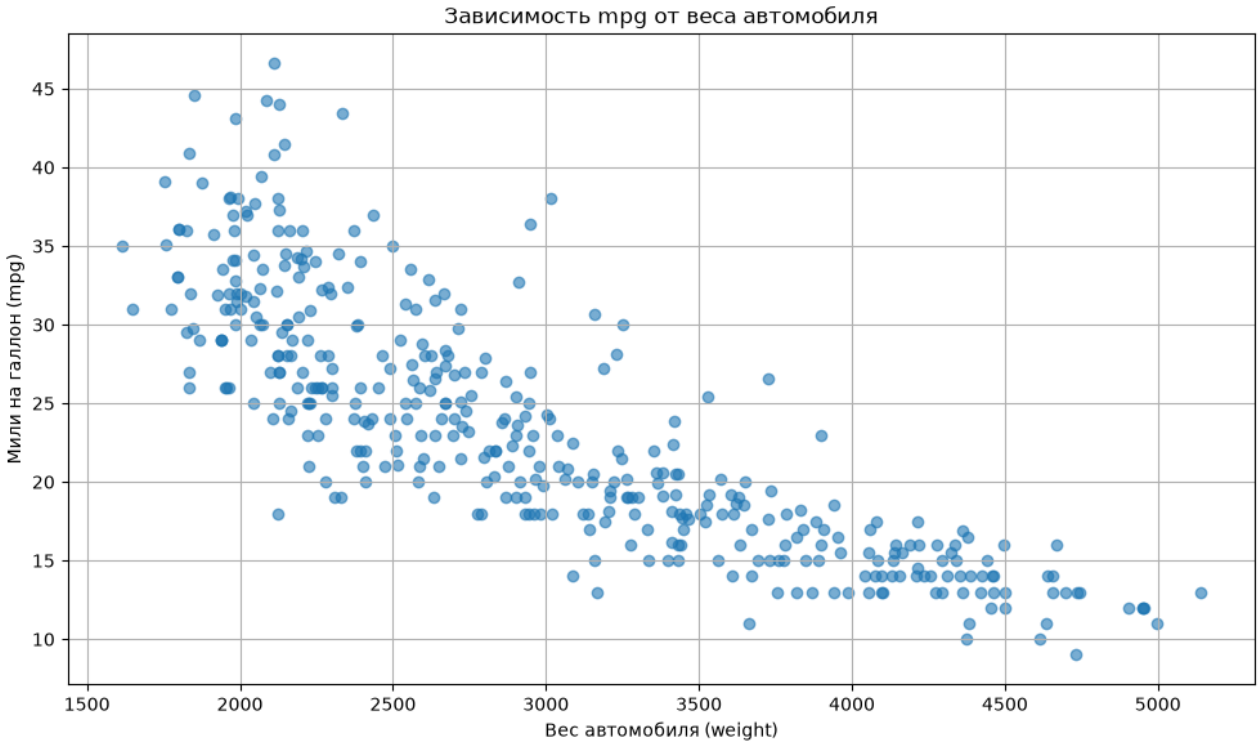
Код программы:

```
df["horsepower"] = pd.to_numeric(df["horsepower"],
errors="coerce")
mean_hp = df["horsepower"].mean()
df["horsepower"] = df["horsepower"].fillna(mean_hp)
print("Среднее horsepower:", round(mean_hp, 2))
```

3. Диаграмма рассеяния mpg от weight

Код программы:

```
plt.figure(figsize=(10, 6))
plt.scatter(df["weight"], df["mpg"], alpha=0.6)
plt.xlabel("Вес автомобиля (weight)")
plt.ylabel("Мили на галлон (mpg)")
plt.title("Зависимость mpg от веса автомобиля")
plt.grid(True)
plt.savefig("01_weight_mpg.png")
print("Корреляция:", round(df["mpg"].corr(df["weight"]), 3))
```



4. Преобразование признака origin в числовой

Код программы:

```
origin_encoded = pd.get_dummies(df["origin"], prefix="origin",
dtype=int)
df = pd.concat([df, origin_encoded], axis=1)
print(origin_encoded.head())
```

5. Новый признак age

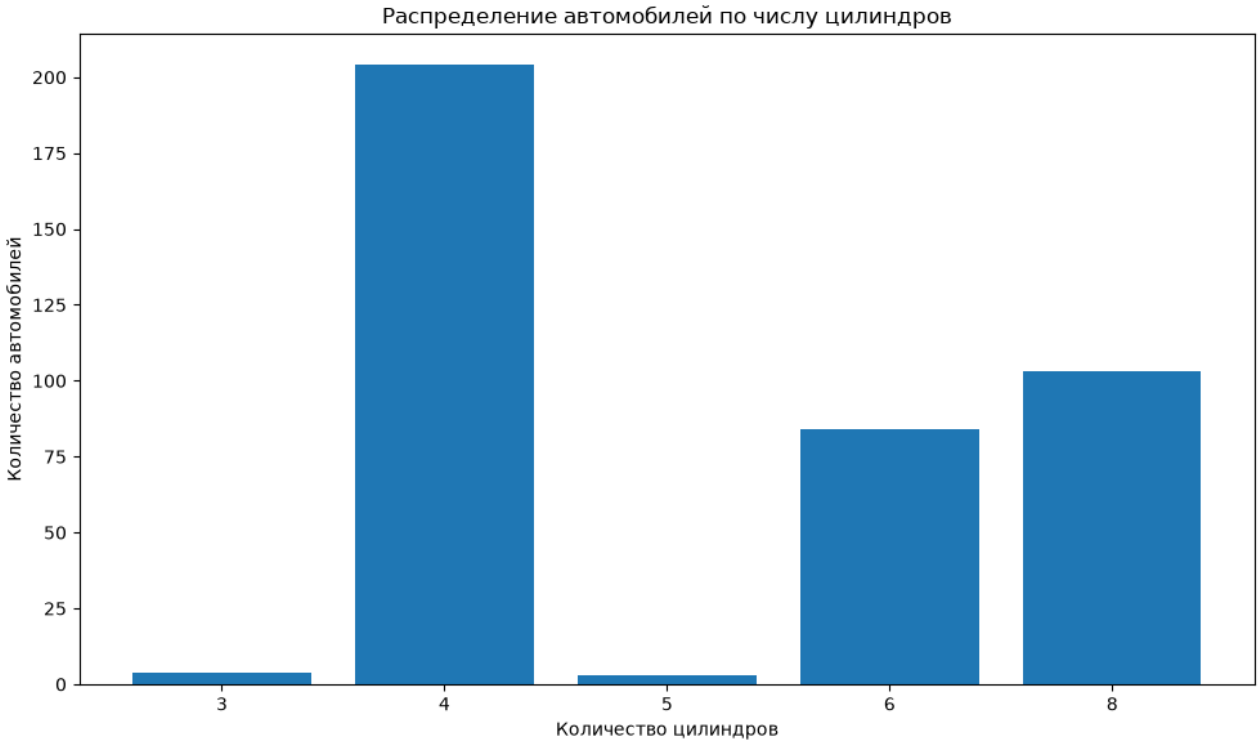
   Код программы:

```
df["age"] = 1983 - (1900 + df["model year"])
print(df[["model year", "age"]].head())
```

6. Распределение количества цилиндров

   Код программы:

```
counts = df["cylinders"].value_counts().sort_index()
plt.figure(figsize=(10, 6))
plt.bar(counts.index.astype(str), counts.values)
plt.xlabel("Количество цилиндров")
plt.ylabel("Количество автомобилей")
plt.title("Распределение автомобилей по числу цилиндров")
plt.savefig("02_cylinders.png")
```



Вывод: В ходе лабораторной работы были получены практические навыки:

- загрузки и анализа данных с помощью **Pandas**;
- обработки пропущенных значений;
- кодирования категориальных признаков методом **One-Hot Encoding**;
- создания нового признака и нормализации числовых признаков;
- визуализации данных с помощью **Matplotlib**.

Датасет Auto MPG был успешно проанализирован: выявлена сильная обратная зависимость расхода топлива от веса автомобиля ($r = -0.832$), установлено, что в выборке преобладают 4-цилиндровые автомобили.